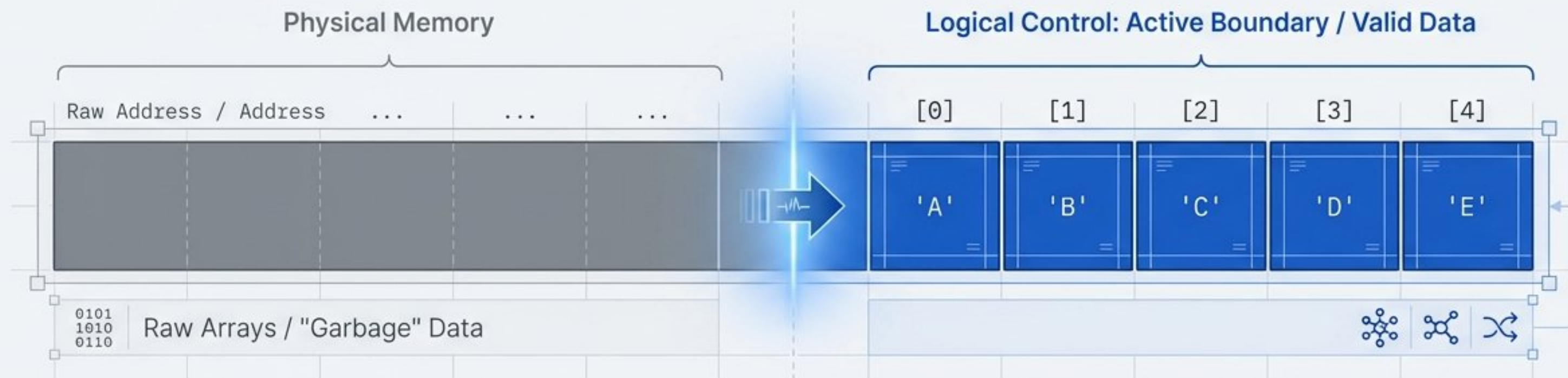




Listas Sequenciais

A Evolução do Armazenamento:
Do espaço físico ao controle lógico.

O que vamos construir hoje



Conceituação: Entender a lista como um Tipo Abstrato de Dados (TAD) construído sobre um vetor estático.

TAD = Estrutura Lógica + Operações sobre um Vetor Físico



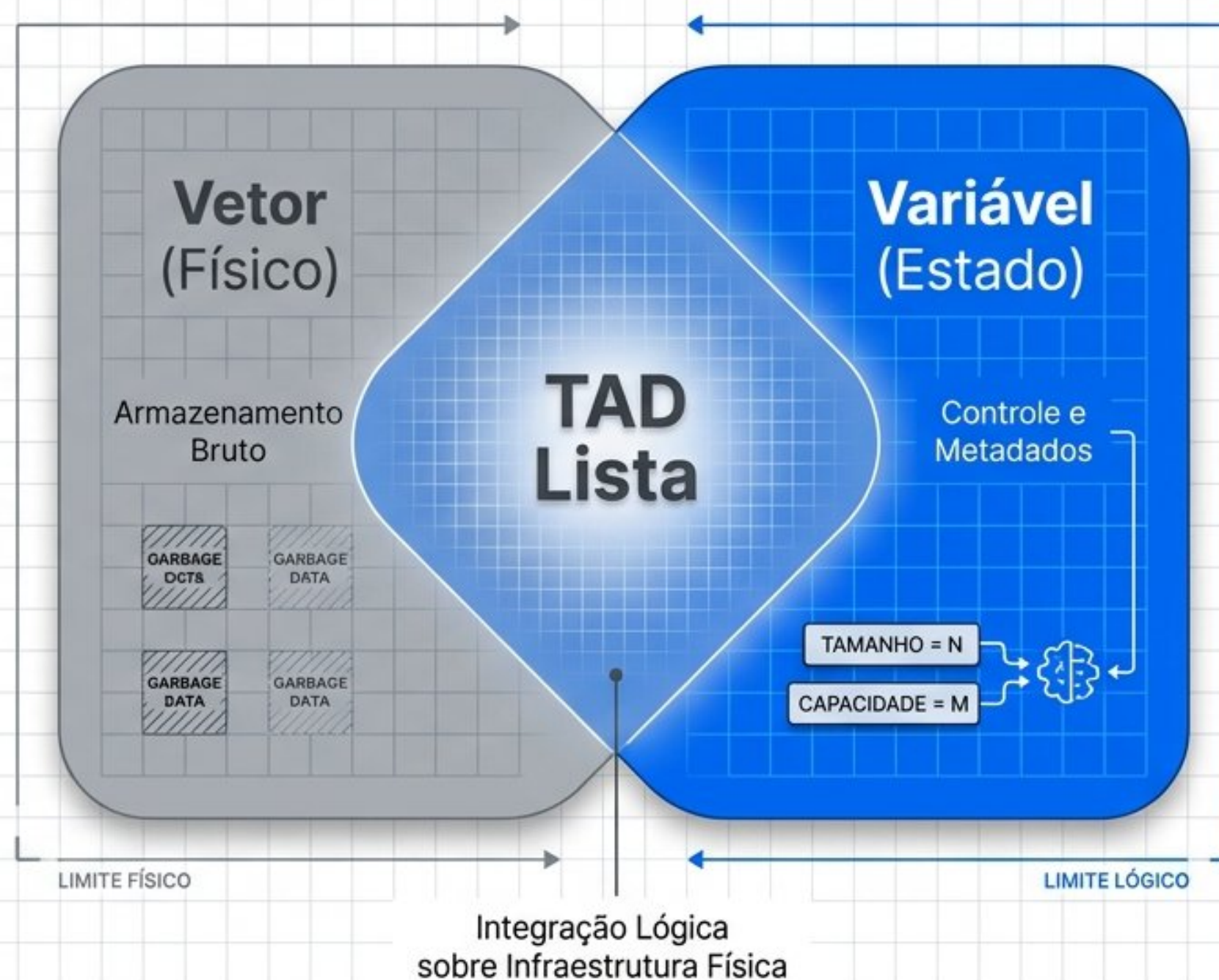
Anatomia: Implementar uma lista sequencial gerenciando 'Capacidade' vs. 'Tamanho Lógico'.



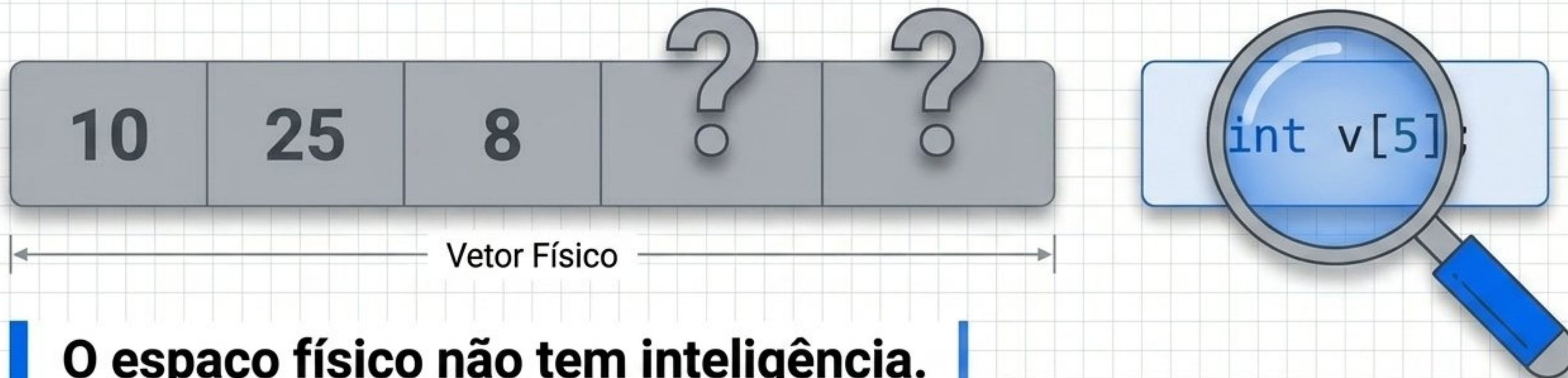
Mecânica: Realizar operações seguras de inserção e remoção no final da estrutura.



Tratamento de Limites: Controlar estados críticos (Lista Cheia e Lista Vazia).



O limite do vetor "cru"



O espaço físico não tem inteligência.

- Um vetor aloca espaço na memória de forma contígua, mas não sabe quantos elementos válidos possui.
- Se inserirmos 3 números em um vetor de 10 posições, como o programa diferencia os "dados válidos" do "lixo de memória"?
- A Necessidade: Precisamos de um mecanismo para gerenciar o estado desse espaço.

O que é uma Lista?



Definição:

Uma coleção ordenada de elementos que pode crescer e diminuir dinamicamente conforme a necessidade da aplicação.

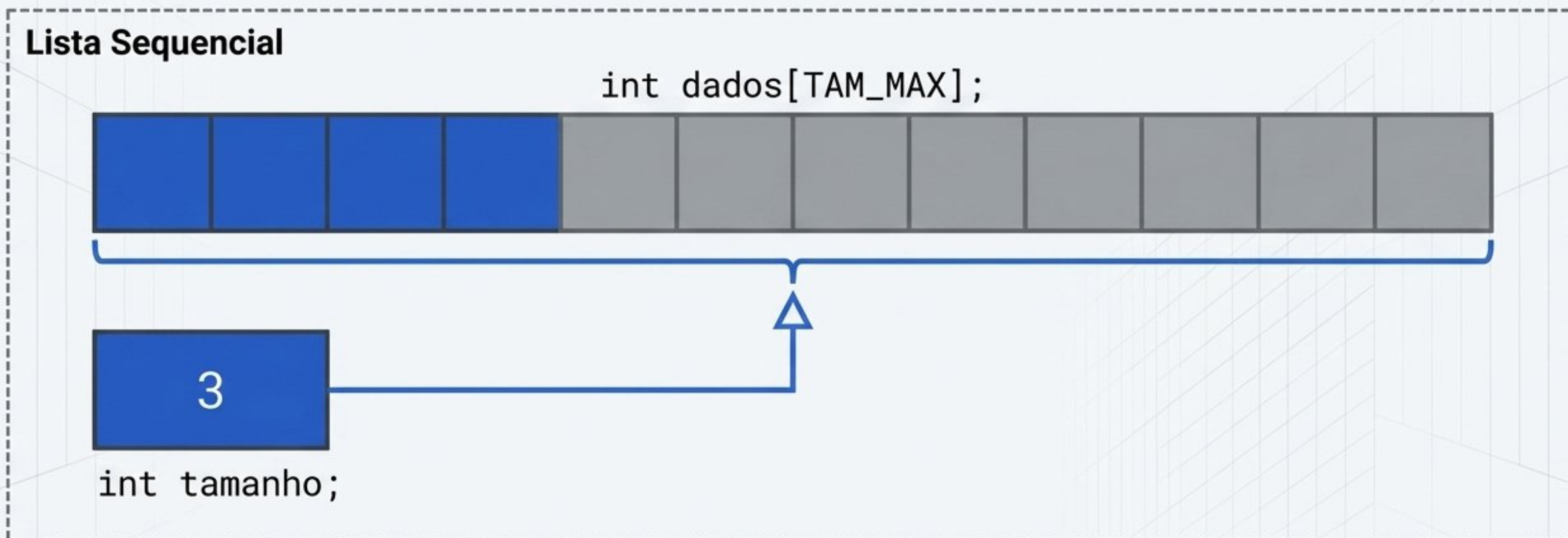
Características (TAD - Tipo Abstrato de Dados):

- Definida por suas operações (Inserir, Remover, Buscar), não apenas pela sua sintaxe.
- Oculta a complexidade interna do usuário (encapsulamento conceitual).

Físico vs. Lógico: A mudança de perspectiva

Vetor Cru (O Físico)	Lista Sequencial (O Lógico)
- Natureza: Estrutura passiva de armazenamento.	- Natureza: Tipo Abstrato de Dados (Vetor + Controle).
- Tamanho: Imutável (TAM_MAX), definido na compilação.	- Tamanho: Elástico (tamanho lógico), varia em tempo de execução.
- Estado: Sem controle; o programa não sabe o que é 'dado válido'.	- Estado: Gerenciado; fronteiras claras entre dados e espaço livre.

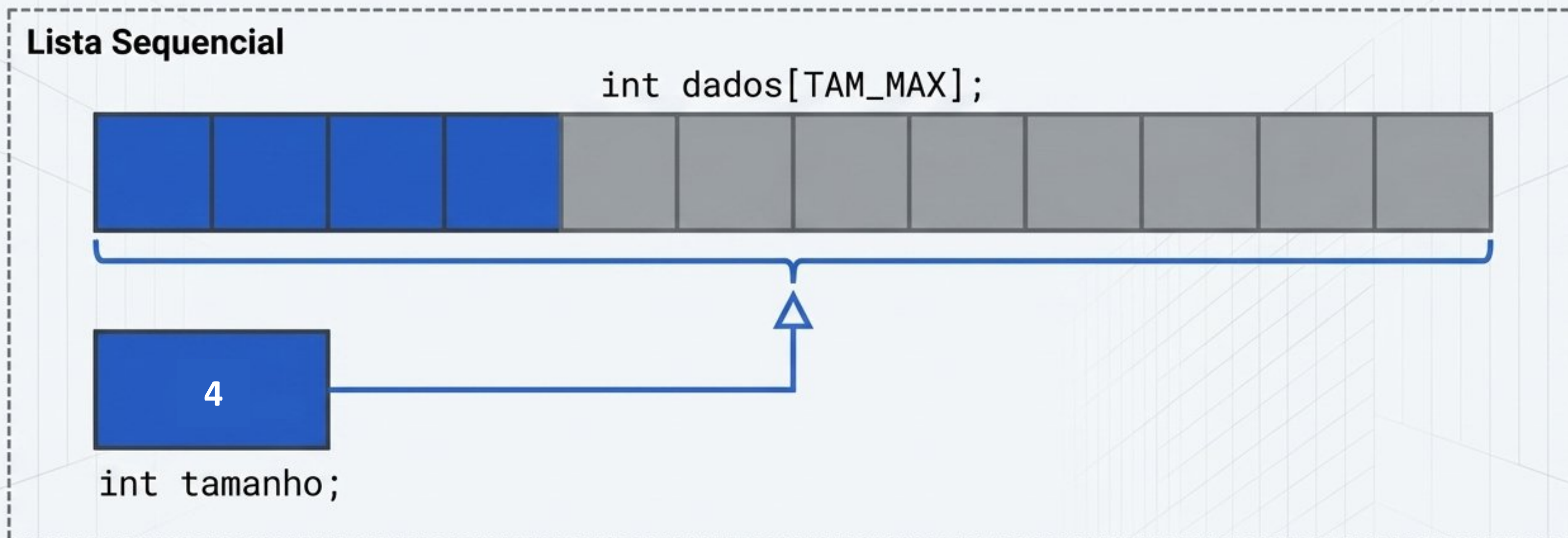
Anatomia da Lista Sequencial



Componente 1: `dados[TAM_MAX]` - O recipiente físico (a capacidade máxima de armazenamento).

Componente 2: `tamanho` - O indicador lógico (a quantidade real de elementos inseridos no momento).

Anatomia da Lista Sequencial

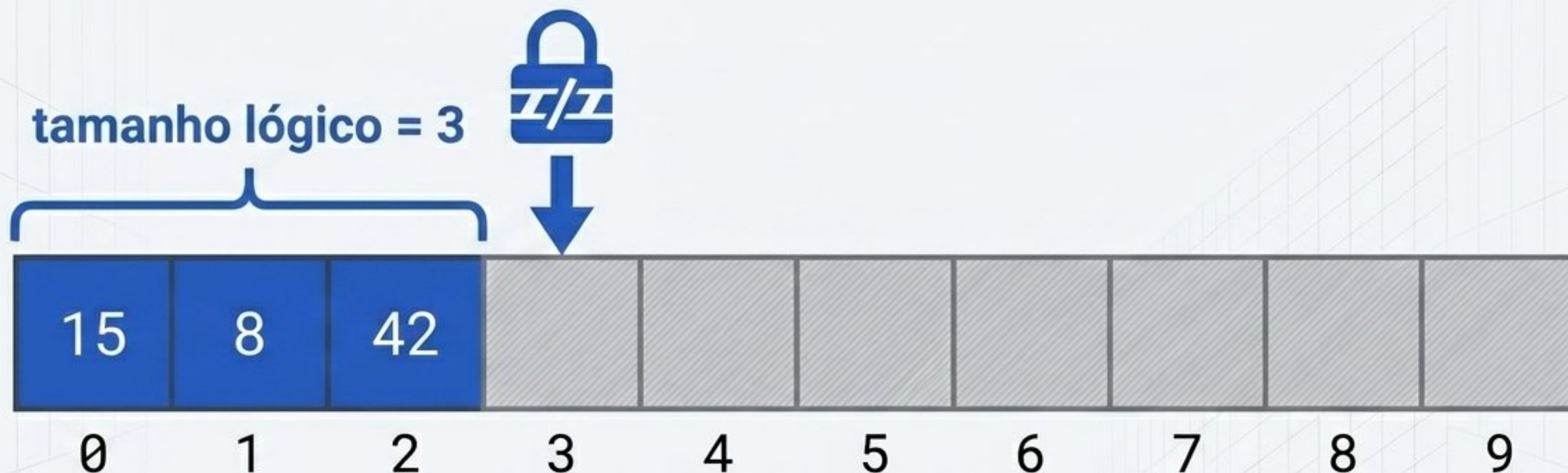


Componente 1: `dados[TAM_MAX]` - O recipiente físico (a capacidade máxima de armazenamento).

Componente 2: `tamanho` - O indicador lógico (a quantidade real de elementos inseridos no momento).

Regra de Ouro: A lista só 'existe' do índice 0 até o índice `tamanho - 1`.

O "Gatekeeper" da Memória

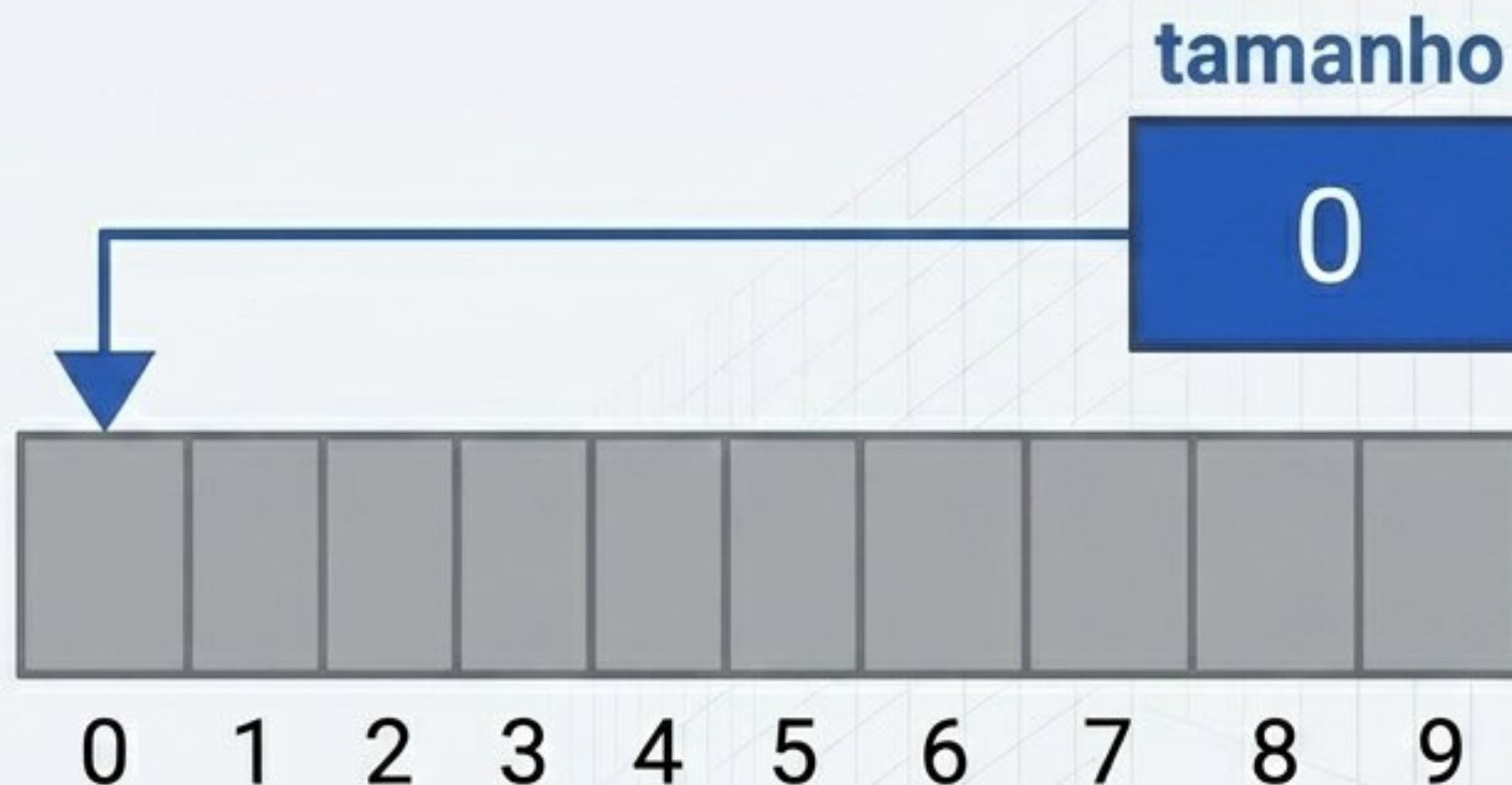


O papel da variável **tamanho**:

- Define a fronteira estrita entre os dados válidos e o espaço disponível.
- Indica o próximo espaço vazio: O valor de **tamanho** é exatamente o índice onde a próxima inserção no fim ocorrerá (neste caso, índice 3).

Operação: Inicializar a Lista

```
void inicializar(int *tamanho)
{
    *tamanho = 0;
}
```



Lógica:

- Para criar (ou esvaziar) uma lista sequencial, não precisamos apagar os dados do vetor físico.
- Basta definir **tamanho = 0**. O espaço físico é instantaneamente 'esquecido' pela camada lógica.

Operação: Inserção no Fim

```
// 1. Inserir no índice 'tamanho'  
dados[tamanho] = novo_valor;  
// 2. Expandir a fronteira lógica  
tamanho++;
```



Validação Crítica: Antes de inserir, o sistema deve sempre verificar se a lista não está cheia ($\text{tamanho} < \text{TAM_MAX}$). Se sim, a operação é rejeitada.

Operação: Remoção do Fim

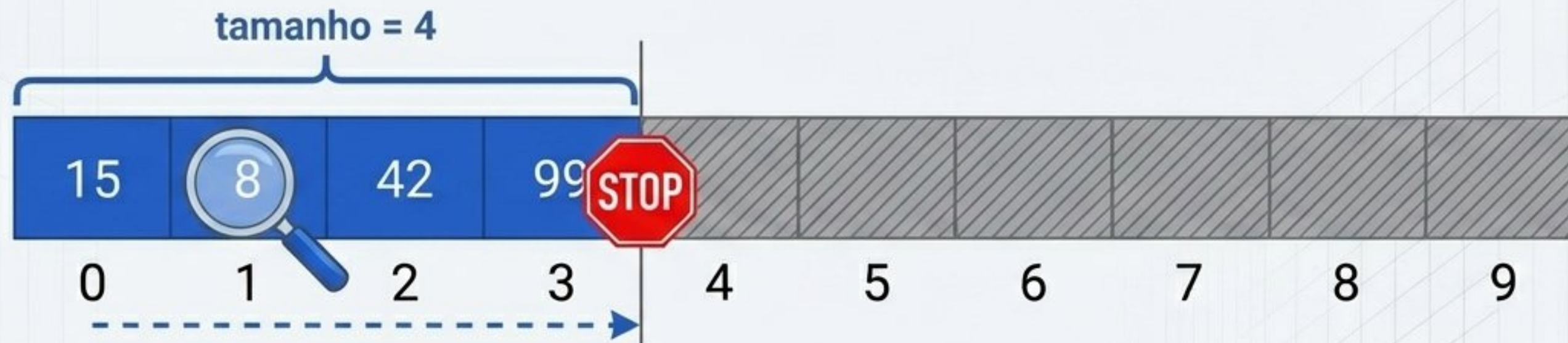
```
// Apenas encolher a fronteira lógica  
tamanho--;
```



A "Mágica" do Lixo de Memória:

- Não apagamos o valor físico do vetor. Apenas decrementamos o tamanho.
- Para a lógica do programa, aquele valor deixou de existir. Na próxima inserção, ele será sobrescrito.
- Validação Crítica: Verificar se a lista não está vazia ($\text{tamanho} > 0$) antes de remover.

Operação: Busca em Lista Sequencial

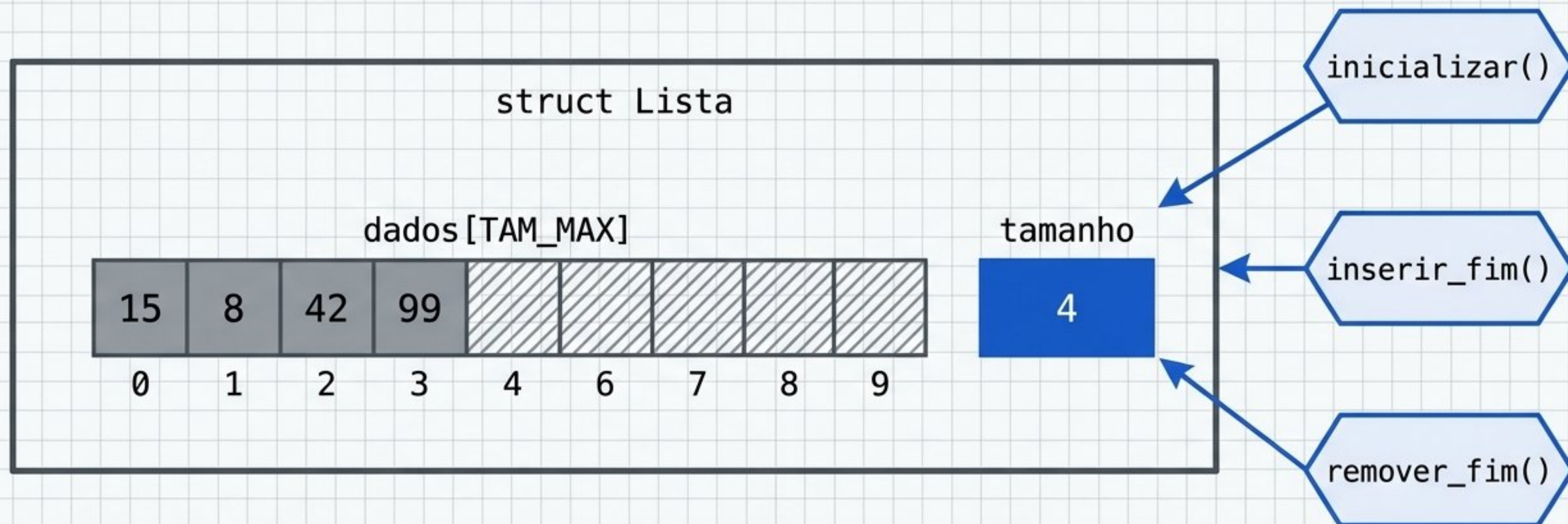


A Lógica Permanece:

- A busca segue exatamente a mesma lógica da Busca Sequencial em vetores (vista na Semana 3).
- A única diferença estrutural é o limite do laço for. Em vez de buscar até o fim da capacidade física, o laço para em $\text{tamanho} - 1$.

```
for (int i = 0; i < tamanho; i++) {  
    if (dados[i] == chave) return i;  
}
```


O Ápice Lógico: Encapsulamento (TAD)



Unificando a Estrutura:

- Em vez de passar o vetor de dados e a variável tamanho separadamente para as funções, nós os unimos.
- Uma **struct** encapsula os dados físicos e a variável de controle lógico em uma única entidade autossuficiente e protegida.
- Este é o verdadeiro poder dos Tipos Abstratos de Dados na engenharia de software.

Prática de Laboratório

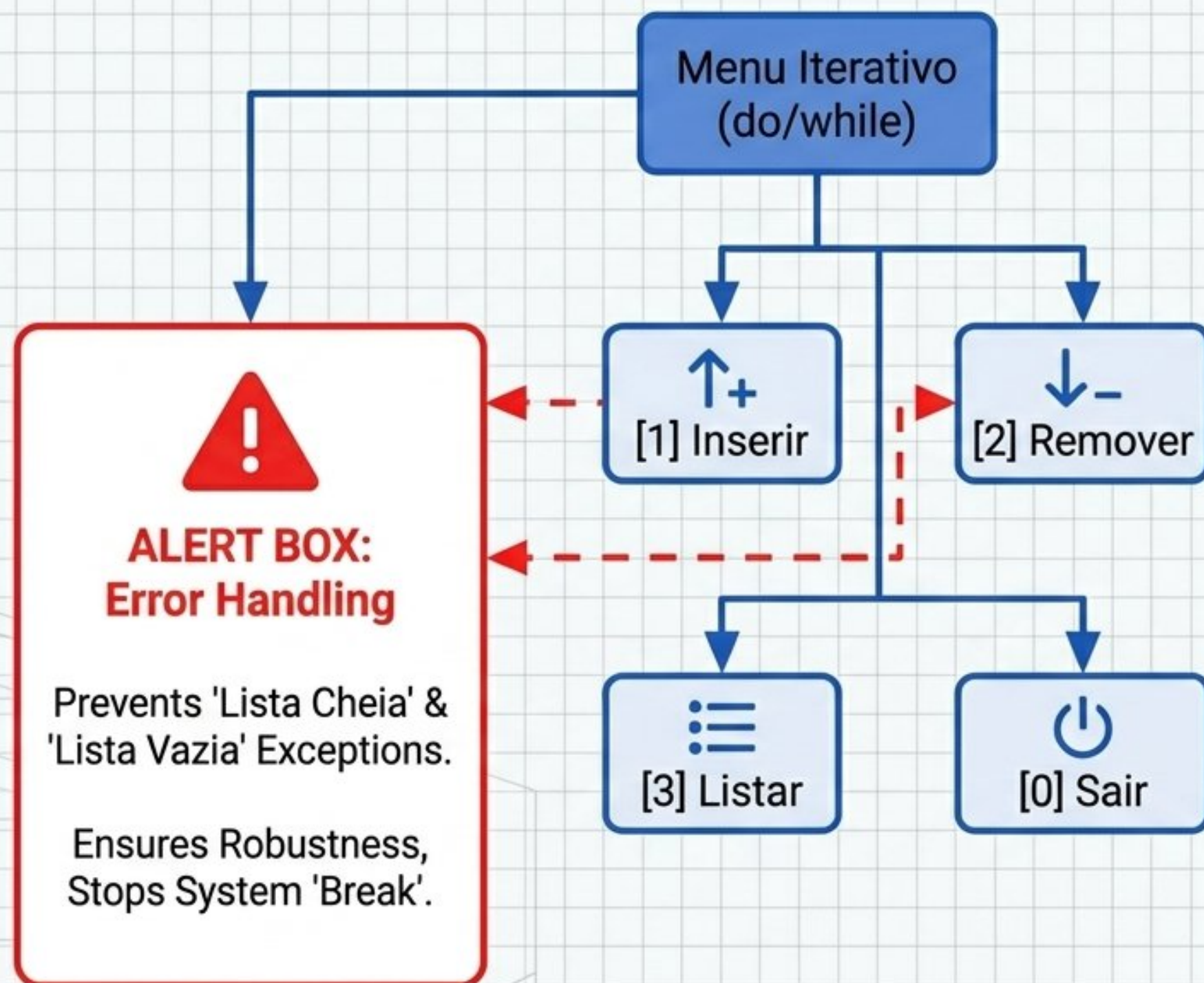
✕ ○ ○

// A Missão:

- Implementar a estrutura da Lista Sequencial utilizando C (vetor + tamanho).
- Criar funções isoladas para inicializar a lista, inserir elementos no fim e remover elementos do fim.
- Criar uma função imprimir_lista que exiba os dados lógicos lado a lado com os valores de capacidade total e ocupação atual.

Nota: Focar na correção das operações e no gerenciamento preciso dos limites (overflow/underflow).

Atividade Verificadora: Sistema Interativo



O Desafio: Desenvolver um programa em C com um menu iterativo.

Requisitos:

1. O menu deve permitir ao usuário: [1] Inserir, [2] Remover, [3] Listar, [0] Sair.
2. Sistema Robusto: O programa não pode 'quebrar'.
3. Deve exibir mensagens claras de alerta caso o usuário tente inserir em uma 'Lista Cheia' ou remover de uma 'Lista Vazia'.
4. Mostrar o estado da lista após cada operação para comprovar o funcionamento da variável de controle.